

# Tokeniser-Aware Structured Shorthand

*The Engineering of Token-Efficient  
Machine-to-Machine LLM Pipelines*

Companion book to the TASS protocol

[github.com/suyash333/TASS-protocol](https://github.com/suyash333/TASS-protocol)

White paper DOI: [10.5281/zenodo.20403219](https://doi.org/10.5281/zenodo.20403219)

Built 2026-07-07 · MIT License

# Tokeniser-Aware Structured Shorthand

---

## *The Engineering of Token-Efficient Machine-to-Machine LLM Pipelines*

Companion book to the TASS protocol · White paper DOI: 10.5281/zenodo.20403219

---

*"The cheapest token is the one you never generate."*

This book is a complete, research-grade treatment of TASS: the economic argument, the tokenizer theory underneath it, the format specification, the reference implementation, and — most importantly — the system-design patterns for deploying it in production. Every code listing runs against the library in this repository; every benchmark is reproducible with the included tooling.

**Who this is for.** Engineers building high-volume LLM extraction pipelines; architects deciding between JSON, TOON, CSV, and compact formats; researchers studying inference-cost optimization; and anyone who has looked at an invoice from an LLM provider and wondered where the money went.

**How to read it.** Chapters 1–3 are the theory and can be read on a train. Chapters 4–8 are the engineering and are best read next to a terminal with `pip install tass-protocol` done. Chapter 9 is the system design chapter — if you are an architect evaluating TASS, you may start there and work backwards. Chapter 10 grounds everything in three complete case studies.

---

## Table of Contents

| #  | Chapter                            | One-line summary                                                                                  |
|----|------------------------------------|---------------------------------------------------------------------------------------------------|
| 1  | The Economics of Output Tokens     | Why output tokens dominate extraction costs, with a cost model you can run                        |
| 2  | Tokenizer Foundations              | BPE mechanics, why <code>~a</code> is cheap and Duployan shorthand is expensive                   |
| 3  | The TASS Format                    | Complete specification: records, grammar, design rationale, comparison to TOON/CSV/protobuf       |
| 4  | Schema Compilation                 | From schema to system prompt; prompt caching as the enabling economics                            |
| 5  | Parsing and Failure Modes          | The parser, type coercion, the failure taxonomy, and defensive <code>safe_parse</code>            |
| 6  | The <code>.tass</code> File Format | <code>@dict</code> / <code>@codes</code> / <code>@records</code> : schemas and data at rest       |
| 7  | Benchmarking Honestly              | Measurement methodology, tiktoken, and the mistakes that inflate savings claims                   |
| 8  | Cryptographic Integrity            | Tamper-evident records with post-quantum-safe symmetric primitives                                |
| 9  | System Design with TASS            | Reference architectures: pipelines, queues, schema registries, fallback ladders, observability    |
| 10 | Case Studies                       | Influencer rates, weather APIs, quantum-device telemetry — end to end                             |
| 11 | TASS Beyond the LLM                | The format on byte- and character-priced channels: IoT/LPWAN, SMS, QR/NFC, logs, URLs, field data |
| A  | API Reference                      | Every public class and function, with signatures                                                  |
| B  | Glossary & Bibliography            | Terms and further reading                                                                         |
| C  | Use-Case Catalog                   | Every use case → its schema, code, and analysis chapter, runnable                                 |

## Runnable companion code

Each snippet is standalone and referenced from the chapters:

| File                                         | Used in      | What it shows                                                                                           |
|----------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------|
| <code>snippets/cost_model.py</code>          | Ch. 1        | Annualized cost model: JSON vs TASS at your volumes                                                     |
| <code>snippets/end_to_end_pipeline.py</code> | Ch. 4, 5, 9  | Complete extraction pipeline: compile → (mock) LLM → parse → validate → sign                            |
| <code>snippets/fallback_ladder.py</code>     | Ch. 5, 9     | The three-rung degradation strategy under malformed output                                              |
| <code>snippets/schema_registry.py</code>     | Ch. 9        | A minimal versioned schema registry with <code>.tass</code> files as the source of truth                |
| <code>snippets/signed_audit_trail.py</code>  | Ch. 8, 9, 10 | The signed firehose end to end: tamper caught, replay deduped, archive re-verified                      |
| <code>snippets/tass_logger.py</code>         | Ch. 11       | TASS as a structured-logging format: stdlib <code>logging.Formatter</code> + expand-on-view + byte bill |
| <code>snippets/payload_budgets.py</code>     | Ch. 11       | Does the record fit? Exact byte costs vs LoRaWAN/SMS/USSD/QR/NFC/URL budgets, signed & unsigned         |

Run any of them from the repository root:

```
pip install -e .
python book/snippets/end_to_end_pipeline.py
```

---

*MIT licensed, like the rest of the repository. Corrections and PRs welcome.*

# Chapter 1 — The Economics of Output Tokens

*In which we establish that the problem is real, quantify it, and derive the conditions under which a compact output format pays for itself.*

## 1.1 The asymmetry nobody prices in

Every LLM API bill decomposes into two line items: input tokens and output tokens. Across every major provider, output tokens cost a multiple of input tokens — historically between 3× and 6×. The asymmetry is not arbitrary pricing; it reflects the physics of inference. Input tokens are processed in parallel in a single forward pass (*prefill*), while output tokens are generated one at a time (*decode*), each requiring a full pass through the model with an ever-growing KV cache. Prefill saturates the accelerator's compute; decode saturates its memory bandwidth and holds the hardware hostage for the duration of generation.

The consequence for pipeline design is blunt:

*Latency and cost both scale with output length, and output length is the one variable the schema designer fully controls.*

You cannot make the model smarter per token. You *can* make every token carry more information.

## 1.2 Where extraction pipelines spend

Consider the canonical high-volume workload TASS targets: fixed-schema extraction. A classifier, a router, a scorer — anything of the shape *"read this text, emit these N fields."* The response is pure structure:

```
{"tier": "micro", "rate_low": 18000, "rate_high": 42000, "gst": false}
```

Tokenize that with `o200k_base` and count what you paid for:

| Token category           | Examples                                                                           | Share of output |
|--------------------------|------------------------------------------------------------------------------------|-----------------|
| Values (the information) | <code>micro</code> , <code>18000</code> , <code>false</code>                       | ~25–35%         |
| Keys (known in advance)  | <code>"tier"</code> , <code>"rate_low"</code>                                      | ~35–45%         |
| Syntax (pure ceremony)   | <code>{</code> , <code>}</code> , <code>"</code> , <code>:</code> , <code>,</code> | ~25–35%         |

Two thirds of the bill purchases nothing. The keys were fixed the moment you designed the schema; the syntax exists so `JSON.parse` doesn't have to think. In a machine-to-machine pipeline — no human ever reads the wire format — this is dead weight at a 4–6× markup.

## 1.3 The information-theoretic floor

There is a floor below which no honest format can go: the entropy of the values themselves. If a field takes one of four values (`nano` | `micro` | `mid` | `macro`), the information content is 2 bits, and the practical floor in a BPE token stream is **one token** — a short code like `mc`. TASS's design goal is stated exactly in these terms:

*Spend approximately one token per value, plus approximately one token per field marker, and zero tokens on syntax.*

For an  $N$ -field record the target is  $\approx 2N + \epsilon$  tokens. JSON for the same record costs  $\approx 6\text{--}9N$ . The ratio — not any particular percentage — is the durable claim, robust across tokenizers, because both numerator and denominator are measured in the same tokenizer.

## 1.4 A cost model you can argue with

Let:

- `C_out` — output price per million tokens
- `V` — calls per day
- `T_json`, `T_tass` — measured output tokens per record in each format

Annual saving:

$$S = 365 \cdot V \cdot (T_{\text{json}} - T_{\text{tass}}) \cdot C_{\text{out}} / 10^6$$

Worked example, deliberately mid-range (`C_out` = \$10/M, a mid-tier 2025 model): the influencer-rate record above measures ~78 tokens as JSON and ~17 as TASS.

| Calls/day | JSON \$/yr | TASS \$/yr | Saved/yr         |
|-----------|------------|------------|------------------|
| 10,000    | \$2,847    | \$620      | <b>\$2,227</b>   |
| 100,000   | \$28,470   | \$6,205    | <b>\$22,265</b>  |
| 1,000,000 | \$284,700  | \$62,050   | <b>\$222,650</b> |

Run `snippets/cost_model.py` with your own schema, volumes, and prices — it measures `T_json` and `T_tass` with the real tokenizer rather than trusting this table.

There is a second, less visible saving: **decode latency**. At a typical 50–150 tokens/second decode rate, dropping 61 tokens per response removes 0.4–1.2 seconds of wall-clock latency per call — often worth more than the money in interactive routing paths.

## 1.5 The costs on the other side of the ledger

A research-grade treatment prices the downsides too. TASS costs you:

1. **A system prompt tax.** The symbol dictionary adds ~50–150 input tokens. With prompt caching (Chapter 4) this amortizes to noise — cached input tokens are typically discounted 90%+ — but on providers without caching, very short conversations, or one-off calls, the tax can exceed the saving. **TASS is a high-volume instrument.**
2. **A reliability tax.** Models are heavily trained to emit JSON; some are constrained-decoded into it. TASS relies on instruction-following. Chapter 5 quantifies the observed failure modes (1–5% depending on model and schema size) and the mitigation ladder that recovers them.
3. **A debuggability tax.** `~t:mc ~l:18k` is hostile to a human reading logs at 2 a.m. Chapter 9's observability section addresses this with expand-on-ingest logging: store compact, view expanded.

## 1.6 The break-even inequality

Putting both sides together, TASS wins when:

$$\begin{array}{ll}
 V \cdot (T_{\text{json}} - T_{\text{tass}}) \cdot C_{\text{out}} & \text{saving per day} \\
 > \\
 V \cdot f \cdot T_{\text{retry}} \cdot C_{\text{total}} & \text{retry overhead per day} \\
 + P_{\text{cache\_miss}} \cdot T_{\text{dict}} \cdot C_{\text{in}} \cdot V & \text{uncached dictionary tax}
 \end{array}$$

where `f` is the malformed-output rate and `T_retry` the cost of a fallback call. With measured values (`f`  $\approx$  2%, retry in JSON mode, cached prompts) the left side dominates by more than an order of magnitude for any schema of 3+ fields. The inequality flips only when volume is low, caching is unavailable, or the schema is so dynamic it changes per call — precisely the "when not to use TASS" list in the README, now derived rather than asserted.

## 1.7 Summary

- Output tokens are 4–6× input price and generated serially; they dominate both cost and latency in extraction workloads.
- In JSON output, roughly two thirds of tokens are keys and syntax whose content is known before the call is made.
- TASS targets  $\approx$  2 tokens per field against JSON's 6–9, a ratio that holds across tokenizers because it is measured, not assumed.
- The savings claim is conditional — high volume, fixed schema, cached prompts — and the break-even condition is explicit and computable.

*Next: why the tokenizer itself dictates the design of the format — Chapter 2.*

## Chapter 2 — Tokenizer Foundations

*In which we open the black box that turns text into money, and derive TASS's alphabet from first principles.*

### 2.1 Byte-pair encoding in five minutes

Modern LLMs do not see characters. They see **tokens**: entries in a fixed vocabulary (~100k–200k entries) learned by byte-pair encoding (BPE). BPE training starts from raw bytes and greedily merges the most frequent adjacent pairs until the vocabulary budget is exhausted. The result is a frequency-ordered compression dictionary of its training corpus:

- `" the"` — one token (astronomically frequent)
- `"rate"` — one token
- `"~"` — one token (common in URLs, paths, code)
- `"fi"` (ligature) — several bytes of UTF-8, byte-fallback tokens

Two properties matter for format design:

**P1 — Frequency is destiny.** A string is cheap iff it (or its pieces) appeared often in the training corpus. ASCII letters, digits, and common punctuation are the cheapest matter in the universe of BPE.

**P2 — Boundaries matter.** Tokenizers merge across what humans consider word boundaries. `"rate_low"` may be 3 tokens (`rate`, `_`, `low`) while `"ratelow"` is 2. Leading spaces are usually part of the token. You cannot reason about token cost by eyeballing; you must measure (Chapter 7).

### 2.2 The stenography hypothesis

TASS's founding observation reframes the tokenizer:

*A BPE vocabulary is a stenographic dictionary the model already knows. Instead of teaching the model new shorthand, choose output strings that already compress to single tokens.*

Classical stenography (Pitman, Gregg, Duployan) maps frequent words to short strokes. BPE has done the same statistically. The format designer's job reduces to: *never emit a string that isn't near the top of the frequency table.*

### 2.3 Why not real stenography symbols? (a cautionary tale)

The obvious romantic idea — use actual shorthand glyphs, e.g. Unicode's Duployan block (U+1BC00–U+1BCAF) — fails empirically, and the failure teaches the design rule. Duployan characters essentially never occur in web-scale corpora, so no BPE merge was ever learned for them. Each glyph falls back to raw UTF-8: **4**

**bytes** → **up to 4 byte-fallback tokens for a single "shorthand" character**. The visually shortest notation is the most expensive. The general rule:

*Token cost is corpus frequency, not visual length. Exotic symbols, emoji, and rare Unicode are anti-compression in a BPE world.*

## 2.4 Deriving the TASS alphabet

Given P1 and P2, score candidate field-marker designs (counts via `o200k_base`; the ordering is stable across `cl100k_base` and open-model tokenizers):

| Candidate marker        | Example                  | Typical tokens                                               | Verdict                         |
|-------------------------|--------------------------|--------------------------------------------------------------|---------------------------------|
| Full JSON key           | <code>"rate_low":</code> | 4–6                                                          | baseline pathology              |
| Bare word key           | <code>rate_low:</code>   | 3–4                                                          | still paying for the name       |
| Two-letter code         | <code>rl:</code>         | 2                                                            | good                            |
| <code>~</code> + letter | <code>~l:</code>         | 2 ( <code>~l</code> merges; <code>:</code> rides with value) | good <b>and collision-proof</b> |
| Duployan glyph          | <code>•</code>           | 3–5                                                          | worst case                      |

Why `~` specifically, rather than nothing at all?

1. **Unambiguous scanning.** A sentinel prefix lets the parser split on whitespace and cheaply discard anything that isn't a field (`pair.startswith("~")` in `tass/parser.py`). Prose contamination, markdown fences, and model apologies are filtered structurally rather than with fragile regex.
2. **Low collision with values.** `~` rarely begins natural-language words, so a value containing spaces or colons can't be mistaken for a field marker.
3. **It is a single frequent token** in every major tokenizer, usually merging with a leading space (`~`) for free.

The symbol pool is `a–z` then `A–Z` — 52 single-token markers (`SchemaCompiler.token_pool`). Beyond 52 fields you are past the sweet spot of a flat record anyway; split the schema (the compiler enforces this with a clear error).

## 2.5 Value encoding under the same rule

The same frequency logic governs values:

- **Enums** → **2-letter codes**. `mc` is one token; `micro` may be one or two; `"micro"` (JSON-quoted) is three. The `@codes` block (Chapter 6) makes the mapping declarative and expansion server-side.
- **Magnitude suffixes.** `18000` tokenizes digit-group by digit-group; `18k` is typically two tokens and parses deterministically (`_expand_k` in the parser: `18k` → `18000`, `1.5k` → `1500`).
- **Booleans** → `0/1`. Single ASCII digits are single tokens; the parser's boolean coercion accepts `1/true/yes`.

- **No quoting, ever.** Quotes are pure syntax; TASS values end at whitespace.

## 2.6 Portability of the argument

A fair objection: "you optimized for one tokenizer." The design deflects it in three ways. First, TASS uses only the *intersection* of what is cheap everywhere — ASCII letters, digits, `~`, `:` — rather than quirks of one vocabulary. Second, the claim is a ratio measured within a single tokenizer, so it survives vocabulary changes. Third, the benchmark harness (Chapter 7) takes `--encoding` precisely so you can re-verify on the tokenizer you actually deploy against. On every vocabulary tested, the ordering `TASS < pipe < T00N < JSON < prose` is preserved even as absolute counts drift.

## 2.7 Summary

- BPE token cost is corpus frequency; visual brevity is irrelevant and exotic symbols are actively harmful (the Duployan lesson).
- TASS's alphabet — `~` + one ASCII letter per field, short ASCII codes per value — is the cheapest *reliable* encoding the intersection of major tokenizers admits.
- The `~` sentinel buys structural robustness (cheap scanning, prose rejection) for one token, which Chapter 5 will spend well.

*Next: the format itself, specified precisely — Chapter 3.*

## Chapter 3 — The TASS Format

*In which the format is specified precisely enough to reimplement from this chapter alone, and positioned honestly among its alternatives.*

### 3.1 The record

A **TASS record** is a single line of `field pairs` separated by single spaces:

```
~t:mc ~f:45 ~e:4.2 ~l:18k ~h:42k ~g:0
```

Each pair is `PREFIX SYMBOL ":" VALUE`:

- **PREFIX** — one character, default `~` (configurable; Chapter 2 gives the selection criteria).
- **SYMBOL** — one ASCII letter `a–z A–Z`, assigned by the schema compiler in declaration order. The reserved symbol `!` carries a cryptographic tag (Chapter 8) and is never assigned to a schema field.
- **VALUE** — any run of non-whitespace characters. Values are untyped on the wire; types live in the schema and are applied at parse time.

#### Grammar (EBNF)

```
record  = pair , { SP , pair } ;
pair    = prefix , symbol , ":" , value ;
prefix  = "~" ;                               (* configurable *)
symbol  = letter | "!" ;                       (* "!" reserved: integrity tag *)
value   = vchar , { vchar } ;                 (* any non-whitespace *)
letter  = "a"..."z" | "A"..."Z" ;
SP      = " " ;
```

Deliberate consequences of this grammar:

- **Order-free.** Keys are explicit, so field order never matters. This buys robustness against models that reorder, and enables canonical hashing (Chapter 8).
- **Self-delimiting.** No closing brace to forget — the most common LLM JSON failure (truncation mid-structure) cannot produce a half-record that silently parses as valid.
- **Unknown-tolerant.** Parsers skip pairs whose symbol is not in the dictionary and tokens that are not pairs. Additive schema evolution is free (Chapter 9 §schema registry).
- **No escaping.** A value cannot contain whitespace. This is the format's sharpest edge, examined in §3.4.

### 3.2 Value conventions

These conventions are contracts between prompt and parser, not grammar:

| Convention         | Wire                           | Parsed        | Mechanism                                       |
|--------------------|--------------------------------|---------------|-------------------------------------------------|
| k-suffix magnitude | 18k, 1.5k                      | 18000, 1500.0 | <code>_expand_k</code> , integer/float fields   |
| Boolean            | 1 / true / yes, 0 / false / no | True / False  | boolean coercion, case-insensitive              |
| Enum code          | mc                             | micro         | <code>@codes</code> table, expanded server-side |
| Float              | 4.2                            | 4.2           | float coercion                                  |

The principle: **the LLM emits the minimum; the server does the arithmetic**. Every expansion that can happen after the paid tokens stop, should.

### 3.3 The format family: an honest comparison

|                                  | JSON                      | TOON   | CSV/pipe      | Protobuf         | TASS              |
|----------------------------------|---------------------------|--------|---------------|------------------|-------------------|
| Tokens / field (measured, o200k) | 6–9                       | 4–6    | 2–3           | n/a <sup>1</sup> | ≈2                |
| Keys on wire                     | full                      | full   | none          | none             | 1 char            |
| Order-sensitive                  | no                        | no     | <b>yes</b>    | no               | no                |
| Truncation detectable            | often silent <sup>2</sup> | varies | <b>silent</b> | yes              | count check       |
| Nesting                          | full                      | full   | none          | full             | none <sup>3</sup> |
| LLM training familiarity         | very high                 | low    | high          | none             | low               |
| Human-readable                   | yes                       | yes    | partly        | no               | with dictionary   |

<sup>1</sup> Binary — LLMs cannot reliably emit it; included as the machine-format reference point. <sup>2</sup> A truncated JSON string can remain syntactically parseable. <sup>3</sup> Flattening conventions cover one level (§3.4).

The interesting rival is **pipe-delimited**, which is nearly as cheap. TASS spends one extra token per field to fix pipe's two production failures: order sensitivity (a model that swaps two columns silently corrupts data; a swapped TASS pair is identical data) and silent field omission (a missing pipe column shifts every subsequent value; a missing TASS field is caught by validation by name). That single token is the best-spent token in the format.

### 3.4 Limits, stated plainly

1. **No whitespace in values.** Encode multi-word values as `@codes` entries (the code travels, the phrase stays server-side). For genuinely free-text fields — summaries, quotes — TASS is the wrong tool; use it for the structured fields and a separate call (or JSON) for prose.
2. **No nesting.** One level of structure can be flattened by convention (`address.city` → its own symbol), and lists of uniform objects become multiple records. Trees and recursive structures should stay in JSON/TOON; this is a boundary, not a bug.
3. **Instruction-following dependency.** Nothing forces a model to emit TASS the way constrained decoding forces JSON. The mitigation ladder in Chapter 5 is therefore not optional equipment — it is part of the protocol.

### 3.5 Design lineage

The record syntax will look familiar to greybeards, deliberately. `key:value` pairs with single-character keys and a sigil prefix echo **RFC 822 headers**, **SI/Metar weather codes**, and amateur-radio exchange formats — all designed under a constraint TASS shares: an expensive, error-prone channel where every character costs and the receiver knows the schema. The novelty in TASS is not the shape but the *costing function*: the channel's unit is the BPE token, and the format is optimized against a measured vocabulary rather than a character count.

### 3.6 Summary

- A TASS record is a flat, order-free, self-delimiting line of `~x:value` pairs; grammar small enough to memorize.
- Types, magnitudes, and enums are schema-side contracts; the wire carries minimal untyped strings.
- Against its nearest competitor (pipe-delimited), TASS pays one token per field to eliminate order-corruption and silent-shift bugs.
- Its limits — no spaces, no nesting, instruction-following dependence — are explicit, and the rest of the book engineers around them.

*Next: turning a schema into a prompt the model actually follows — Chapter 4.*

## Chapter 4 — Schema Compilation

*In which a plain dictionary becomes a system prompt, and prompt caching turns a fixed cost into a rounding error.*

### 4.1 The compiler contract

`SchemaCompiler` (in `tass/parser.py`) is deliberately tiny. It takes a flat schema — field names to type hints — and returns two coupled artifacts:

```
from tass import SchemaCompiler

parser_map, system_prompt = SchemaCompiler().compile({
    "user_intent":      "string",
    "urgency_level":    "integer",
    "requires_routing": "boolean",
})
```

- **system\_prompt** — injected verbatim into every LLM call.
- **parser\_map** — `{symbol: (field_name, type_hint)}`, fed to `TASSParser` on the receiving side.

The two artifacts are generated in one pass from one source, which is the point: **the prompt and the parser can never disagree about the dictionary**, because neither is written by hand. Symbols are assigned positionally (`a`, `b`, `c`... from a 52-slot pool); exceeding the pool raises immediately with instructions to split the schema.

### 4.2 Anatomy of the generated prompt

```
You are a data extraction engine.
Output ONLY the TASS format below. No prose. No markdown. No explanation.
Format: ~a:<value> ~b:<value> ~c:<value>
Dictionary:
~a = user_intent
~b = urgency_level
~c = requires_routing
```

Each line earns its tokens:

1. **Role framing** ("data extraction engine") — moves the model out of conversational register, measurably reducing prose contamination.
2. **The triple prohibition** ("No prose. No markdown. No explanation.") — targets the three observed contamination modes separately. Models that ignore one prohibition often obey another.
3. **Format: line** — a *positive* template. Models imitate examples far more reliably than they follow abstract rules; this line is the single highest-value line in the prompt.

4. **Dictionary:** — grounds each symbol in the semantic field name, so the model's own understanding of "urgency\_level" transfers to `~b`.

## Hardening for smaller models

The compiler's output is a floor, not a ceiling. Two additions measurably improve compliance on small open models, at modest prompt cost:

- **One-shot example.** Append `Example: ~a:refund ~b:5 ~c:true`. A concrete record outperforms any additional rule.
- **Value constraints in the dictionary comments** — e.g. `~b = urgency_level (integer 1-5)`. Constraint violations drop sharply when the range is stated adjacent to the symbol.

## 4.3 Prompt caching: the enabling economics

The TASS dictionary is a fixed prefix repeated across thousands of calls — the textbook prompt-caching workload. Every major provider now offers prefix caching with cached-input discounts of roughly 90%.

The arithmetic for a 3-field schema ( $\approx 70$ -token dictionary), 100k calls/day, at illustrative prices (\$3/M input, 90% cache discount):

|                                     | Uncached | Cached        |
|-------------------------------------|----------|---------------|
| Dictionary tokens/day               | 7.0M     | 7.0M          |
| Effective price                     | \$3.00/M | \$0.30/M      |
| Dictionary cost/day                 | \$21.00  | <b>\$2.10</b> |
| vs. output tokens saved/day (Ch. 1) | ~\$61    | ~\$61         |

Uncached, the dictionary consumes a third of the saving; cached, 3%. Hence the design rule:

***Keep the dictionary byte-stable.** Any change — reordered fields, a renamed comment, trailing whitespace — invalidates the cache prefix. Generate the prompt once from the schema, version it (Chapter 9), and never string-format per-request values into the system prompt. Dynamic content belongs in the user message, after the cached prefix.*

## 4.4 Schema design guidelines

Field-level choices compound at volume:

1. **Order fields by stability, most stable first** — not for the wire (TASS is order-free) but for the cache: schema evolution that appends fields preserves the shared prompt prefix.
2. **Prefer enums to free strings.** Every field you can close over a code table (`@codes`, Chapter 6) removes both tokens and a failure mode (Chapter 5's "value type deviation").

3. **One record, one decision.** If two field groups are populated by logically separate reasoning (say, sentiment fields and billing fields), two small schemas beat one wide one — smaller dictionaries, independent failure domains, better cache granularity.
4. **Name fields for the model, not the database.** The dictionary line `~b = urgency_level` is *prompt text*; `urgency_level` primes the model better than `urg_lvl_v2`. Map to internal names after parsing.

## 4.5 The end-to-end loop

The complete lifecycle, runnable as `snippets/end_to_end_pipeline.py`:

```
from tass import SchemaCompiler, TASSParser

schema = {"user_intent": "string", "urgency_level": "integer",
          "requires_routing": "boolean"}

parser_map, system_prompt = SchemaCompiler().compile(schema)
parser = TASSParser(dictionary_map=parser_map)

# response = llm(system=system_prompt, user=ticket_text)  # your provider
response = "~a:refund ~b:5 ~c:true"                      # (mock)

result = parser.safe_parse(response)
# {'user_intent': 'refund', 'urgency_level': 5, 'requires_routing': True}
```

Compile once at deploy time; parse per call. The compiler never appears on the hot path.

## 4.6 Summary

- One schema compiles into a coupled (prompt, parser-map) pair, making prompt/parser drift structurally impossible.
- The generated prompt's lines each target a specific, observed failure mode; a one-shot example is the cheapest upgrade for small models.
- Prompt caching reduces the dictionary tax  $\sim 10\times$ , and preserving it imposes one discipline: a byte-stable system prompt with all dynamic content in the user message.

*Next: what comes back, and everything that can be wrong with it — Chapter 5.*

## Chapter 5 — Parsing and Failure Modes

*In which we treat the LLM as an unreliable transmitter, catalogue every way its output goes wrong, and build the receiver that survives all of them.*

### 5.1 The adversarial framing

The correct mental model for parsing LLM output is a noisy channel. The transmitter is probabilistic, was trained mostly on *other* formats, and occasionally decorates its output with apologies. A production parser is therefore built like a protocol receiver: scan defensively, coerce liberally, validate strictly, and always have a fallback.

TASSParser implements this in four layers, each independently testable.

### 5.2 Layer 1 — Cleaning (`_strip_markdown`)

The most common contamination is cosmetic: code fences and surrounding prose.

Sure! Here is the extraction:

```
~a:refund ~b:5 ~c:true
```

The cleaner strips fence markers, then — crucially — selects **the first line that starts with the prefix character**. The `~` sentinel from Chapter 2 pays its rent here: prose lines cannot begin with a field pair, so line selection is structural, not heuristic.

### 5.3 Layer 2 — Scanning (`_parse_line`)

The scanner splits on whitespace and applies three filters per token: must start with the prefix; must contain `:`; symbol must exist in the dictionary. Everything else is *silently skipped* — stray words, unknown symbols, the `~!` integrity tag (Chapter 8). This is Postel's law applied deliberately: **liberal in scanning, strict in validation** (layer 4 catches what leniency lets through).

### 5.4 Layer 3 — Coercion (`_coerce`)

Wire values are untyped strings; the schema's type hints drive coercion: integers and floats pass through `_expand_k` (`18k` → `18000`, `1.5k` → `1500`), booleans accept `1/true/yes` case-insensitively, strings pass through untouched. Coercion *absorbs* the "value type deviation" failure class: a model that writes `~c:True` or `~b:5.0` where an int was expected still parses.

## 5.5 Layer 4 — Validation (`_validate`)

Validation checks that every schema field is present, by name, and raises `TASSValidationError` listing exactly what is missing. This is where the one-token-per-field key investment (Chapter 3) matures: unlike positional formats, a missing field is *identifiable*, and a partial record is *usable* if your application can tolerate specific absences.

## 5.6 The failure taxonomy

Observed failure classes, their frequency envelope (varies by model tier and schema width), and their handling:

| #  | Failure                                                       | Typical rate     | Caught by                        | Recovery                    |
|----|---------------------------------------------------------------|------------------|----------------------------------|-----------------------------|
| F1 | Markdown fences / prose wrapper                               | common, cosmetic | Layer 1                          | free                        |
| F2 | Value type deviation ( <code>True</code> , <code>5.0</code> ) | 2–5%             | Layer 3                          | free                        |
| F3 | Missing field(s)                                              | 1–3%             | Layer 4                          | retry or partial-accept     |
| F4 | Unknown symbol invented                                       | <1%              | Layer 2                          | skipped; F3 catches the gap |
| F5 | Model answers in JSON anyway                                  | ~1%              | <code>safe_parse</code> fallback | free                        |
| F6 | Unparseable prose                                             | <0.5%            | all layers fail                  | retry ladder                |

F5 deserves note: models trained hard toward JSON sometimes revert to it under pressure (long context, unusual input). `safe_parse` treats this as *success in the wrong dialect* — it attempts JSON after TASS fails and returns the same dict shape:

```
result = parser.safe_parse(raw)  # TASS → validate → JSON → raise
```

## 5.7 The fallback ladder

Individual mitigations compose into a degradation strategy — runnable as `snippets/fallback_ladder.py`:

|                                      |                                      |
|--------------------------------------|--------------------------------------|
| Rung 1: <code>safe_parse(raw)</code> | ~97–99% resolve here                 |
| Rung 2: one retry, same TASS prompt  | transient noise; ~half the rest      |
| Rung 3: retry in explicit JSON mode  | pay full JSON price this once        |
| Rung 4: dead-letter queue + alert    | input is pathological; a human looks |

Two properties make the ladder economically sound. First, expected cost: at `f`  $\approx$  2% rung-2 rate, the overhead is  $0.02 \times T_{\text{tass}}$  extra tokens per record on average — negligible against the 60–80% baseline saving. Second, **the ladder is also your monitor**: rung transition rates are the health metric for the whole system (a rising rung-2 rate is how you learn a provider silently swapped model versions — Chapter 9 §observability).

## 5.8 What the parser refuses to do

Boundaries, stated as design decisions:

- **No schema inference.** The parser never guesses types from values; ambiguity belongs to the schema author, not runtime.
- **No partial-field repair.** A mangled pair (`~a refund`) is dropped, not "fixed" — repairing corrupt fields invents data. F3 validation reports the gap honestly instead.
- **No semantic validation.** That `urgency_level ≤ 5` is business logic; it lives above the parser (see the pipeline snippet), keeping the library's contract purely structural.

## 5.9 Summary

- Treat LLM output as a noisy channel; the parser is a four-layer receiver: clean, scan, coerce, validate.
- Six failure classes cover practice; four are absorbed for free by design, and the remaining two are handled by a four-rung fallback ladder whose expected cost is ~2% overhead.
- The ladder doubles as the system's health instrumentation — rung rates are the metric that catches silent model drift.

*Next: schemas and records at rest — the `.tass` file — Chapter 6.*

## Chapter 6 — The `.tass` File Format

*In which the wire format acquires a home on disk, and the schema becomes a versionable artifact.*

### 6.1 Why a file format at all

The wire format of Chapters 3–5 is ephemeral — it exists between the model's decoder and your parser. But three durable artifacts orbit it: the **dictionary** (symbols → fields), the **code tables** (short codes → full values), and often the **records themselves** (compact storage is storage savings too). The `.tass` file gives all three one human-review-able, diff-able, git-friendly container:

```
@dict
  ~t = tier           # mc=micro | md=mid | mk=macro
  ~l = rate_low      # INR, k suffix = thousands
  ~g = gst           # 0 = no | 1 = charge 18%
@end

@codes
  mc = micro
  md = mid
  mk = macro
@end

@records
~t:mc ~l:18k ~g:0
~t:md ~l:55k ~g:1
@end
```

The design borrows deliberately from formats that survived decades: INI-style sections, `#` comments stripped before parsing, and one record per line so that `grep`, `wc -l`, `tail -f`, and line-oriented diffs all work unmodified. **A `.tass` file is a spec you can code-review.**

### 6.2 The three blocks

| Block                 | Maps                                                     | Consumed by                       |
|-----------------------|----------------------------------------------------------|-----------------------------------|
| <code>@dict</code>    | <code>~symbol = field_name</code>                        | prompt generation + parser map    |
| <code>@codes</code>   | <code>code = full value</code> (spaces allowed in value) | server-side expansion after parse |
| <code>@records</code> | one wire-format record per line                          | storage / test fixtures / replay  |

Block structure is flat: a block opens with `@name`, closes with `@end`, and unknown block names are a hard error (`TASSFileError`) — silent tolerance of a typo'd `@dcit` would swallow an entire dictionary. Content *outside* any block is ignored, which is what makes the generous comment headers in `spec/` and `examples/` legal.

## 6.3 The codes table as a semantic contract

`@codes` is where TASS's "expand after the paid tokens stop" principle becomes declarative. The LLM emits `~c:lr`; the parser looks up `lr` in the codes table and delivers `Light Rain` to the application. Three properties worth internalizing:

1. **Codes are global to the file, not per-field.** `mc = micro` applies to any field whose value is `mc`. Keep code namespaces disjoint per file (tier codes, template codes, status codes must not collide) — a discipline, not an enforcement, in format 1.0.
2. **Unmatched values pass through.** A value with no code entry is returned verbatim. Numeric fields therefore need no entries, and code tables can be introduced incrementally.
3. **Multi-word expansion is free.** `lr = Light Rain` costs the wire nothing; the phrase exists only server-side. This is the sanctioned answer to "TASS values can't contain spaces."

## 6.4 Parsing files

```
from tass import TASSFileParser

tfile = TASSFileParser().parse_file("examples/weather.tass")

tfile.dictionary    # {'t': 'tier', ...}
tfile.codes         # {'mc': 'micro', 'lr': 'Light Rain', ...}
tfile.records       # [{'tier': 'micro', ...}] — codes already expanded
tfile.raw_records   # original wire lines, preserved for hashing/signing

parser_map = tfile.to_parser_map()  # feed straight into TASSParser
```

Note the deliberate redundancy of `records` vs `raw_records`: expanded dicts serve the application; raw lines serve cryptography (Chapter 8 MACs are computed over canonical wire form, so files must preserve it) and replay testing.

One honest limitation: `@dict` entries carry no type hints in format 1.0, so `to_parser_map()` defaults every field to `string`. Pipelines that need typed parsing from a file should keep the schema-as-code path (Chapter 4) authoritative and treat `.tass` files as data + documentation. Typed dictionary entries are the natural format 1.1 extension.

## 6.5 The file as source of truth

Because a `.tass` file contains everything both sides of the wire need, it can anchor a deployment workflow:

```
git repo:  schemas/ticket_routing_v3.tass    ← reviewed, versioned, tagged
          |
          ├── CI: parse file → generate system prompt → publish to registry
          └── CI: to_parser_map() → bundle with consumer service
```

Prompt and parser are now built from one reviewed artifact per version — the same never-disagree property as Chapter 4's compiler, extended across services and time. Chapter 9 §9.4 builds the full registry pattern on this foundation, with `snippets/schema_registry.py` as the working miniature.

## 6.6 Summary

- The `.tass` file packages dictionary, code tables, and records in one line-oriented, comment-friendly, git-native container.
- `@codes` turns multi-word values into a server-side concern and makes value semantics reviewable.
- `raw_records` preservation keeps files compatible with the integrity layer; type hints are the acknowledged 1.1 gap.
- A versioned `.tass` file can serve as the single source of truth from which prompts and parser maps are both derived.

*Next: measuring all of this without fooling yourself — Chapter 7.*

## Chapter 7 — Benchmarking Honestly

*In which we measure token savings without fooling ourselves — and catalogue the ways benchmark numbers get inflated.*

### 7.1 The only valid measurement

There is exactly one defensible way to compare formats: **render the same records in each format and count tokens with the tokenizer of the model you will actually call**. Everything else — character counts, "roughly 4 chars per token" folklore, eyeballing — produces numbers that do not survive contact with a real vocabulary (Chapter 2 P2: boundaries are unintuitive).

The repository's harness does precisely this:

```
tass benchmark examples/weather.tass          # tiktoken, o200k_base
tass benchmark examples/weather.tass --encoding cl100k_base
tass benchmark --schema schema.json --records records.json
```

`TASSBenchmark` renders each record four ways — TASS, minified JSON, pipe-delimited, prose — and reports absolute tokens plus percentage against the JSON baseline, with per-format samples printed so the reader can verify the renderings are fair.

### 7.2 The seven ways to lie with a token benchmark

A checklist for reading (or publishing) any format-savings claim. Each item is a mistake that inflates numbers, and the harness's corresponding defense:

1. **Pretty-printed baseline.** Indented JSON adds whitespace tokens the API would never emit. *Defense:* the harness renders JSON with `separators=(",", ":")` — minified, the strongest reasonable baseline.
2. **Cherry-picked field names.** Long snake\_case keys flatter any key-eliding format. *Defense:* benchmark **your** schema, not the author's demo. The harness takes arbitrary schemas for this reason.
3. **Ignoring the prompt tax.** Output savings quoted without the dictionary's input cost. *Defense:* Chapter 1's break-even inequality; the tax belongs in any end-to-end claim (with and without caching).
4. **Ignoring the retry tax.** A 2% fallback-to-JSON rate shaves ~1–2 points off net savings. *Defense:* multiply it in (Ch. 5 §5.7).
5. **Single-record samples.** Token counts are jumpy at small N; one record's percentage is noise. *Defense:* benchmark whole record sets; the harness aggregates across all records in the file.
6. **Wrong tokenizer.** Savings measured on `cl100k_base`, deployed against a different vocabulary. Ratios are *similar* across tokenizers, not identical. *Defense:* `--encoding` — measure what you deploy.
7. **Conflating estimate with measurement.** The harness's `--approx` mode (whitespace/punctuation splitting, for offline use) is deliberately labelled `approx (whitespace)` in its output. Approximate numbers are for direction, never for publication.

## 7.3 Reading the numbers in this repository

Applying the checklist to the repo's own claims, as a worked example of the discipline:

- The README's headline table (17 vs 78 tokens, ↓78%) is a **single 4-field record** — legitimate as an illustration, item 5 says don't quote it as *the* number.
- The weather case study (↓92% vs the full API response) compares against **nested provider JSON including fields the schema drops**. That is the honest number for "replace this API-relay pipeline with TASS" but not for "TASS vs minified flat JSON of the same fields" — the latter is ↓69–73%. Both numbers are real; they answer different questions. **State which question you are answering.**
- The quantum telemetry example (↓69% vs flat JSON, 6 records) is the most conservative and most generalizable figure in the repo.

The honest generic claim, then: **60–80% output-token reduction versus minified flat JSON on typical 4–12 field schemas, before the (small, cache-dependent) prompt and retry taxes**. Larger figures require the nested-baseline framing and should say so.

## 7.4 Beyond static counts: compliance benchmarking

Token counts are static analysis. The dynamic question — *does model M reliably emit TASS for schema S?* — requires a live harness: send N representative inputs, run the full parse-validate pipeline, report the rung distribution of Chapter 5's ladder. The repository's roadmap tracks this as the 100-call cross-model harness. Until your own numbers exist, budget with the envelope from Chapter 5 ( $f \approx 1\text{--}5\%$ ), and re-run the compliance benchmark on every model version change — silent model upgrades are the largest source of compliance drift in production.

## 7.5 Summary

- Count tokens with the deployment tokenizer over rendered records; everything else is folklore.
- Seven specific inflation mistakes cover most bad benchmark claims; the harness defends against them structurally where it can.
- This repository's own numbers, audited: ↓60–80% is the defensible general claim; ↓90%+ requires (and must declare) the nested-API baseline.
- Static token counts must be paired with dynamic compliance measurement, re-run on every model change.

*Next: making records tamper-evident — Chapter 8.*

## Chapter 8 — Cryptographic Integrity

*In which records acquire tamper evidence, and every primitive choice is justified against both classical and quantum adversaries.*

(Companion reference: `docs/CRYPTO.md` — the operational threat model. This chapter covers the reasoning.)

### 8.1 Why integrity is TASS's problem

TASS's target workload — high-volume, machine-to-machine, no human ever reads the wire — is precisely the workload where silent corruption thrives. A flipped digit in a JSON invoice a human reviews gets caught; a flipped digit in record 384,112 of a calibration stream does not. And the records travel:

```
LLM → parser (trust boundary) → queue → storage → analytics, months later
```

Every hop after the parser is an opportunity for bugs, misconfigured consumers, or attackers to alter data. The integrity layer (`tass/crypto.py`) lets the first trusted hop attach a MAC so every later hop — or an auditor long after — can verify nothing changed.

One boundary before anything else, because it is the most common design error in LLM-adjacent cryptography:

*Never ask the model to emit its own signature. A model cannot keep a secret key; its output is attacker-influenceable text. Signing happens at the first trusted hop — your parser — after validation, never before.*

### 8.2 Canonicalization: hashing meaning, not bytes

TASS is order-free (Chapter 3), so two byte-different lines can be the same record. MACs over raw bytes would make semantically identical records verify differently — a reordering hop (a queue that re-serializes, a proxy that normalizes whitespace) would break every tag. The layer therefore MACs the **canonical form**: keep only `~key:value` pairs, drop the signature field itself, sort by key, join with single spaces.

```
canonicalize("~b:2 ~a:1 ~!:oldtag") # → "~a:1 ~b:2"
```

Corollaries: field reordering and whitespace changes never invalidate a tag; `hash_record()` (SHA3-256 of canonical form) gives every record a stable content address usable for deduplication and audit logs.

## 8.3 The primitive choices, and the quantum question

The layer uses three primitives, all symmetric, all stdlib:

| Role                   | Primitive                                | Why                                                      |
|------------------------|------------------------------------------|----------------------------------------------------------|
| Hash / content address | SHA3-256                                 | modern margin, distinct lineage from SHA-2               |
| Authentication         | HMAC-SHA3-256, truncated to 128-bit tags | NIST SP 800-107-approved truncation; compact on the wire |
| Key derivation         | HKDF (RFC 5869) over SHA3-256            | context-separated per-pipeline keys                      |

The quantum reasoning, spelled out once, carefully:

- **Shor's algorithm** breaks RSA and elliptic-curve cryptography outright on a cryptographically relevant quantum computer. The layer contains **no public-key material at all**, so Shor has nothing to attack. This is post-quantum safety *by construction*, not by adopting new math.
- **Grover's algorithm** provides at most a quadratic speedup on brute-force search, reducing a 256-bit key search to  $\sim 2^{128}$  quantum operations — beyond any projected feasibility, and even that ignores Grover's poor parallelization. This is the same reasoning under which NIST classifies AES-256 and SHA-3 as quantum-resistant.
- **Tag truncation** to 128 bits does not help a quantum forger: forgery against HMAC requires either the key (Grover-hard) or a lucky guess ( $2^{-128}$  per attempt, online, rate-limitable).

What symmetric MACs *cannot* give is **non-repudiation** — any key holder can create valid tags, so a MAC proves "someone with the key signed this," never "specifically Alice signed this." When third parties must verify without holding the secret, the correct tool is a post-quantum signature scheme — ML-DSA (FIPS 204) or SLH-DSA (FIPS 205) — layered at the application level. That is the roadmap's PQ-signature item, and the canonical form defined here is deliberately reusable as its message input.

## 8.4 The in-band tag

The tag travels *inside* the record as a `~!:` pair:

```
~t:mc ~l:18k ~h:42k ~g:0 ~!:3vX9kQ2mP8sT1uW4yZ6bCg
```

`!` sits outside the `a-zA-Z` symbol pool, so no schema field can ever collide with it, and Chapter 5's scanner skips it as an unknown symbol — **a signed record parses identically to an unsigned one** (test-enforced: `test_signed_record_still_parses`). The 16-byte tag encodes to 22 base64url characters; signed records need no envelope, no sidecar metadata, and survive any TASS-clean transport.

## 8.5 Key hygiene in three rules

1. **Derive, don't share.** `derive_key(master, context=b"pipeline-v1")` — HKDF context separation means a key extracted from one pipeline's config is useless against another's records.

2. **Rotate by context.** Bump the context string ( `...-v2` ); old records verify under the old derived key, which you keep for audit.
3. **Verification is constant-time** ( `hmac.compare_digest` ) — the library refuses to leak tag prefixes through timing, so keep it that way: never pre-compare tags with `==` "as a fast path."

And the boundaries, restated from the threat model: the layer provides integrity only — **no confidentiality** (TLS or payload encryption for secret values), **no replay protection** (put a timestamp/nonce in the schema; the MAC covers it, your consumer checks it), and **no proof of LLM correctness** (a valid tag means unaltered-since-signing, not extracted-correctly).

## 8.6 Summary

- Sign at the first trusted hop, never inside the model.
- MACs cover a canonical form, so semantically identical records verify identically; the same form doubles as a content address.
- Symmetric-only primitives are post-quantum-safe by construction (no target for Shor; Grover leaves  $\geq 128$ -bit margins).
- The in-band `~!` tag makes signed records self-contained and fully backward-compatible with every parser in this book.

*Next: assembling all eight chapters into production architectures — Chapter 9.*

## Chapter 9 — System Design with TASS

*In which the previous eight chapters assemble into architectures: where TASS belongs in a system, the reference patterns, and the operational practices that keep it healthy.*

### 9.1 The placement principle

Every architecture in this chapter is a variation on one rule:

*TASS lives on exactly one link: from the model's decoder to the first trusted parser. Upstream of that link is prompt territory; downstream is normal typed data. The rest of your system should not know TASS exists.*

Corollaries: never propagate raw TASS strings past the parse boundary (downstream services consuming wire format couples them to symbol assignments); never let a second service re-parse; and expansion, validation, and signing all happen at that single boundary.

### 9.2 Pattern I — The synchronous extraction service

The base pattern; everything else extends it. Runnable miniature: `snippets/end_to_end_pipeline.py`.

Architecture diagram (Mermaid source — renders on GitHub)

```
flowchart LR
    subgraph deploy["deploy time (once)"]
        S["schema vX"] --> C["SchemaCompiler"]
        C --> P["system prompt<br/>byte-stable, cached"]
        C --> M["parser map"]
    end
    subgraph hot["hot path (per call)"]
        U["input text"] --> L["LLM<br/>cached prefix + input"]
        L -- ".cached prefix." --> P
        P --> R["TASSParser<br/>safe_parse + validate"]
        R --> B["business rules"]
        B --> O["typed record out"]
    end
```

Design notes:

- **Compile at deploy, not per request** — the compiler exists off the hot path; the prompt must stay byte-stable for the cache (Ch. 4).
- **The parse boundary is one function** wrapping `safe_parse` + `validate` + business rules; give it a single owner and a single metrics emitter.

- **Semantic checks live above the parser** (Ch. 5 §5.8): range checks, cross-field constraints, allowed-enum enforcement — after coercion, before anything consumes the record.

## 9.3 Pattern II — The asynchronous firehose

At volume, extraction runs behind a queue, and a decision appears that does not exist in Pattern I: **what format crosses the queue?**

Architecture diagram (Mermaid source — renders on GitHub)

```

flowchart LR
    W["W[workers × N<br/>LLM + parse + sign] -->|signed TASS lines| Q[(queue / log<br/>e.g. Kafka)]"]
    Q --> V["V[consumer:<br/>verify → expand → type]"]
    V --> DB["DB[(warehouse)]"]
    Q --> A["A[audit archive<br/>raw signed lines]"]
  
```

Ship **signed raw TASS lines** on the queue, and expand at consumption. The reasoning:

- A TASS line is 3–5× smaller than its expanded JSON — queue throughput and retention costs shrink by the same factor as the LLM bill.
- The MAC (Ch. 8) is computed over canonical wire form; keeping the wire form on the queue means every consumer, and the audit archive, can verify independently. Expansion first would destroy verifiability.
- This is the *sanctioned exception* to §9.1's "don't propagate raw TASS": the queue consumer is a second trusted parser, fed by contract (same schema version, same key context), not by accident. The schema registry (§9.4) is what makes that contract explicit.

Idempotency comes free: `hash_record()` of the canonical line is a natural dedup key for at-least-once delivery.

## 9.4 Pattern III — The schema registry

The moment two services or two schema versions exist, symbol assignments become a coordination problem (`~c` means `requires_routing` in v2 and `confidence` in v3 — a silent-corruption bug waiting to happen). The fix is the standard one — a registry — with `.tass` files as the artifact (Ch. 6). Working miniature: `snippets/schema_registry.py`.

```

schemas/                                     ← a git repo; review = schema governance
  ticket_routing/
    v1.tass
    v2.tass                                 ← append-only field evolution
  weather_ingest/
    v1.tass
  
```

Rules that keep it sound:

1. **Versions are immutable.** A published `vN.tass` never changes; evolution is a new file. (Cache-friendliness falls out: immutable schema ⇒ byte-stable prompt ⇒ permanent cache prefix.)

2. **Producers pin, consumers tolerate.** Each LLM worker pins one version; consumers accept a version *range*, which TASS's unknown-symbol tolerance (Ch. 3) makes cheap — a v1 consumer reading v2 records simply skips the new fields.
3. **Additive-only within a major version.** New fields take fresh symbols; renaming or re-typing a symbol is a new major version and a new key context (Ch. 8) — cryptographic separation enforcing the social rule.
4. **Every record carries its version** — one extra field (`~z:3` by convention, ~2 tokens) buys unambiguous replay forever. At firehose scale, message-envelope metadata can carry it instead.

## 9.5 Pattern IV — The fallback ladder as circuit breaker

Chapter 5's ladder, promoted to an architectural component (`snippets/fallback_ladder.py`):

Architecture diagram (Mermaid source — renders on GitHub)

```

flowchart TD
    A[safe_parse] -->|ok ~98%| Z[accept]
    A -->|fail| B[retry same prompt]
    B -->|ok| Z
    B -->|fail| C[retry in JSON mode]
    C -->|ok| Z2[accept + flag dialect_fallback]
    C -->|fail| D[dead-letter + alert]
    B -. "rung-2 rate > threshold" .-> E["circuit breaker:<br/>switch pipeline to JSON mode,<br/>page owner"]
  
```

The promotion is the dashed edge: rung-transition rates, aggregated, form a **circuit breaker**. If rung-2 entries exceed a threshold (say 5% over 10 minutes), the pipeline as a whole degrades to JSON mode and pages a human. You pay full JSON prices during the incident, but you never stop serving. The two conditions that trip it in practice — a silent provider model swap, or a schema/prompt deploy mismatch — are exactly the ones you want paged on anyway. The breaker converts TASS's main structural risk (instruction-following dependence, Ch. 3) from an outage mode into a cost blip.

## 9.6 Observability: store compact, view expanded

The debuggability tax (Ch. 1) is repaid with tooling, not format changes:

- **Log raw lines, view expanded.** Persist the compact signed line (cheap, verifiable); make dashboards and `kubectl logs pipelines run tass read / expansion` at *view* time. Humans see field names; storage sees 17 tokens.
- **The four golden signals for a TASS pipeline:** rung distribution (ladder health), field-missing rate *per field* (F3 localized — one flaky field usually means one bad dictionary line), MAC verification failure rate (should be ~0; nonzero is an incident, not a metric), and output tokens per record (drift = model started chattering).
- **Trace the boundary.** The parse function from §9.2 is one span: attach `hash_record()` as an attribute, and every downstream anomaly can be traced to the exact wire line that produced it.

## 9.7 The decision framework

Compressing Chapters 1–8 into the questions an architect asks, in order:

| # | Question                                                      | If no →                                                                          |
|---|---------------------------------------------------------------|----------------------------------------------------------------------------------|
| 1 | Fixed schema, repeated $\geq 10^4 \times$ /day?               | Stay with JSON; savings won't clear the taxes (Ch. 1)                            |
| 2 | Machine-to-machine (no human reads the wire)?                 | Stay with JSON/TOON; readability is worth the tokens                             |
| 3 | Flat-ish records ( $\leq 1$ nesting level, $\leq 52$ fields)? | JSON/TOON for the tree parts; TASS only for flat sub-records (Ch. 3)             |
| 4 | Provider offers prompt caching?                               | Re-run the break-even inequality; TASS may still win at high volume (Ch. 1 §1.6) |
| 5 | Can you deploy the fallback ladder + breaker?                 | Do not ship TASS without it (\$9.5)                                              |

Five yeses → Pattern I; add the queue at scale (II), the registry at two schemas (III). The patterns are cumulative, and each is independently removable if its precondition disappears.

## 9.8 Summary

- One placement rule — TASS lives on the decoder → parser link — and four patterns: synchronous service, signed-firehose queue, `.tass`-file schema registry, and the fallback ladder promoted to circuit breaker.
- The queue pattern ships signed wire lines and expands at consumption, preserving both compactness and verifiability.
- Registry rules (immutable versions, pin/tolerate, additive-only, version field) turn symbol assignment from a footgun into governance.
- Observability inverts the debuggability tax: store compact, view expanded, and alert on rung rates — which also catch silent model swaps.

*Next: three complete deployments, end to end — Chapter 10.*

## Chapter 10 — Case Studies

---

*In which three real schemas exercise everything in this book, each chosen to stress a different part of the design.*

All three live in the repository and parse today:

```
tass read spec/sample.tass --records-only
tass read examples/weather.tass --records-only
tass read examples/quantum_telemetry.tass --records-only
```

### 10.1 Influencer rate cards — the origin workload

**File:** `spec/sample.tass` · **Stresses:** value-convention design, the economics at business scale.

The schema that motivated TASS: 10 fields per influencer profile — tier, follower count, engagement, three rate points, two multipliers, GST flag, template id — extracted thousands of times daily at a bootstrapped startup, where the LLM bill was a top-three cost line.

What to study in the file:

- **Every value convention from Chapter 3 in one record.** `~t:mc` (enum code), `~l:18k` (magnitude suffix), `~n:1.2` (bare float), `~g:0` (boolean digit), `~p:fmm` (code that stays a code — the template id is resolved by a *different* service, so the codes table documents it without expanding it).
- **Units live in the dictionary comments**, not on the wire: `~f = followers # integer in thousands` — `~f:45` means 45,000. The comment is documentation *and* prompt text (Ch. 4): one line serving both audiences.
- The economics: at 78 → 17 tokens and this workload's volume, Chapter 1's model prices the format change as a mid-five-figure annual saving — the difference between "optimization" and "runway."

### 10.2 Weather ingestion — the API-relay pattern

**Files:** `examples/weather_raw.json` → `examples/weather.tass` · **Stresses:** schema *reduction*, codes tables at full power, honest baselines.

The input is a provider-style weather response: ~60 fields, four levels of nesting, Kelvin temperatures, internal station ids. The TASS schema is the result of asking, field by field, "*does any consumer read this?*" — 21 fields survive, flattened, unit-converted (K → °C, m/s → km/h) by the pre-processing layer so the wire carries decision-ready values.

What to study:

- **Three codes namespaces in one file** — conditions (`lr = Light Rain`), 16-point compass (`SW = Southwest`), WHO AQI labels (`4 = Poor`) — kept collision-free by construction (Ch. 6 discipline: conditions are lowercase pairs, compass is uppercase, AQI is digits).
- **Multi-word expansion** doing real work: the wire never carries a space; consumers still receive `"Broken Clouds"`.
- **The two honest numbers** (Ch. 7): `↓92%` against the full nested API response — the right figure for "replace this relay pipeline" — and `↓69–73%` against minified flat JSON of the same 21 fields, the right figure for format-vs-format claims. The file's own comment block quotes both, with the baseline named.

This case is the template for the most common TASS adoption path: *an LLM already summarizes an upstream API; move the contract from "emit JSON like the API's" to "emit these N fields in TASS."*

## 10.3 Quantum-device telemetry — the integrity workload

**File:** `examples/quantum_telemetry.tass` · **Stresses:** precision handling, signing, the firehose pattern.

Superconducting-qubit calibration: every few hours, every qubit on a device reports T1/T2 coherence times, gate error rates, readout error, frequency, anharmonicity, and a status code. A 100-qubit machine emits tens of thousands of identical-schema records weekly — and downstream *compilers* consume them to choose which qubits run which circuits, so a silently corrupted `~s:0.00021` is not a logging bug, it is a wrong answer on hardware.

What to study:

- **Precision as schema contract.** Gate errors carry 5 decimals, T1/T2 one decimal, frequency four — stated in the dictionary comments, so the prompt communicates precision expectations and reviewers can check them (Ch. 4 §4.2 hardening).
- **Status codes model a lifecycle** (`ok/dg/fl/rt` → calibrated/ degraded/failed/retuning) — enums earning their two characters, since a consumer routing on `fl` must never fuzzy-match prose like `"calibration failed (see log)"`.
- **This is Pattern II from Chapter 9 end to end:** sign each line at the parser (`tass sign`, key context `quantum-cal-v1`), ship signed lines on the queue, verify at the warehouse, archive raw lines for audit. `hash_record()` dedups the at-least-once queue. The MAC's post-quantum rationale (Ch. 8) is pleasingly non-decorative in a quantum lab.

```
export TASS_KEY="lab-master-secret"
tass sign "~q:0 ~d:hr ~a:312.4 ~s:0.00021 ~g:ok" --context quantum-cal-v1
tass verify "<signed line>" --context quantum-cal-v1 # exit 0
```

### 10.4 What the three cases teach jointly

|                           | Rates                             | Weather                    | Telemetry                |
|---------------------------|-----------------------------------|----------------------------|--------------------------|
| Schema width              | 10                                | 21                         | 11                       |
| Records source            | LLM extraction                    | LLM over API relay         | pipeline + LLM summaries |
| Codes tables              | 2                                 | 3                          | 2                        |
| Savings (honest baseline) | ↓78% (illustrative single record) | ↓69–73% flat / ↓92% nested | ↓69%                     |
| Distinctive lesson        | value conventions                 | schema reduction           | integrity at scale       |

The convergence across three unrelated domains — the savings landing in the same 60–80% band predicted by Chapter 1's  $\approx 2$ -vs-6–9 tokens-per-field analysis — is the empirical spine of this book: the ratio is a property of the *format*, not of any flattering example.

*Appendices follow: the API reference, and a glossary with further reading.*

# Chapter 11 — TASS Beyond the LLM

*In which the format outgrows its origin story. The tokenizer motivated TASS; nothing in the format requires one.*

## 11.1 What TASS actually is, once you remove the LLM

Strip away Chapters 1–2 and examine what remains: a **flat, order-free, self-delimiting, single-line, pure-ASCII record format with out-of-band key and value dictionaries and in-band authentication**. That property list is valuable on any channel where one or more of these units is scarce:

| Scarce unit              | Channel examples                    | §    |
|--------------------------|-------------------------------------|------|
| Bytes on air             | LoRaWAN, NB-IoT, satellite, APRS    | 11.2 |
| Characters per message   | SMS, USSD, pager, chain memos       | 11.3 |
| Pixels / modules         | QR codes, NFC tags, barcodes        | 11.4 |
| Storage × retention      | structured logs, audit archives     | 11.5 |
| URL length & readability | deep links, webhook state           | 11.6 |
| Human attention          | field data collection, ops runbooks | 11.7 |

One framing change first. In LLM pipelines the budget unit was the token; here it is the **byte or character**, and the honest comparison set changes with it: the competitor is no longer JSON alone but binary formats — CBOR, protobuf, custom bitfields — which will *always* beat TASS on pure bytes. TASS's claim on these channels is therefore never "smallest possible" but a specific trade:

*Within ~1.5–2× of binary size, TASS stays printable, grep-able, diff-able, hand-typable, and debuggable over a serial console — with no schema compiler, no codegen step, and no special tooling on either end.*

Where that trade is wrong (firmware-to-firmware links with mature protobuf toolchains, multi-megabyte payloads), use the binary format. The sections below are the channels where it is right.

## 11.2 Constrained telemetry: IoT, LPWAN, and remote sensing

**The constraint.** LoRaWAN payloads run 51–222 bytes depending on data rate; every byte costs airtime, battery, and regulatory duty-cycle budget. Satellite IoT (Swarm-class) bills per ~192-byte packet.

**The fit.** A fixed-schema sensor record is the LLM-extraction workload with the transmitter swapped: same fields every time, receiver knows the schema, nobody reads the wire — except that *somebody occasionally does*, over a serial console at a muddy field site, which is exactly when a printable format saves the day.

`examples/iot_sensor.tass` implements an agricultural soil-sensor fleet:

```
~n:a3 ~t:23.4 ~m:31 ~p:6.8 ~b:87 ~s:ok
```

38 bytes for node id, temperature, moisture, pH, battery, and status — inside even the 51-byte LoRa DR0 budget, against 93 bytes as minified JSON (measured by `payload_budgets.py`). A CBOR encoding runs ~28 bytes; the ~10-byte premium buys `grep '~s:lo' fleet.log` working on the raw capture and a technician reading packets off a UART with no tooling.

Two channel-specific practices:

- **The dictionary ships in the device docs, not the packet** — the `.tass` file (Ch. 6) is the interface contract between firmware and ingest, versioned in git like any registry schema (Ch. 9 §9.4).
- **Signing at the gateway, not the node** (Ch. 8's trust-boundary rule transplanted): constrained nodes use the link layer's native crypto; the gateway — the first hop that speaks TCP — signs the TASS line before it enters your queue, and Pattern II (Ch. 9 §9.3) proceeds unchanged.

### 11.3 Character-budget messaging: SMS, USSD, and chain memos

**The constraint.** One SMS segment is 160 GSM-7 characters; exceed it and cost doubles and delivery atomicity vanishes. USSD sessions cap around 182. Blockchain memo fields are harsher still (Stellar: 28 bytes text). These channels are ASCII-hostile to binary formats — base64 inflates binary by 33% and survives being *typed by a human* poorly.

**The fit.** TASS is already in the channel's native alphabet. A mobile money confirmation:

```
~x:p ~a:1450 ~c:KES ~r:254712345678 ~i:TX88412 ~s:ok
```

52 characters — one third of an SMS segment, leaving room for a `~!` tag (78 signed) so the receiving system can authenticate the confirmation *through* an untrusted SMS aggregator. The same record as JSON is 102 characters and one added field away from a second segment. `snippets/payload_budgets.py` computes exact fits for every budget in this chapter, signed and unsigned.

This is also where TASS's ancestry (Ch. 3 §3.5) completes its circle: METAR and amateur-radio exchange formats solved character-priced channels a century ago with the same move — short codes plus a shared dictionary. TASS is that move with a validation-and-signing layer.

## 11.4 Optical and near-field: QR codes and NFC tags

**The constraint.** QR capacity is quantized: a shorter payload means a lower QR version, physically larger modules, and better scans at distance, at angle, and on damaged labels. NTAG213 NFC stickers hold ~144 bytes total.

**The fit.** Asset tags, event tickets, lab-sample labels — flat records with a known schema, read by machines, occasionally squinted at by humans. A signed equipment tag:

```
~i:EQ-4471 ~k:pump ~l:B2 ~d:2026-03 ~!:hV2nQ8rT1mW4yZ6b
```

~60 bytes → QR version 3 with medium error correction, and the `~!` tag makes the *sticker itself* tamper-evident: a photocopied tag with an edited location fails `tass verify` at the scanner. The alternative — URL-to-database-row — requires connectivity at scan time; the TASS tag carries its payload and its proof offline. (Deep-link hybrids belong in §11.6.)

## 11.5 Structured logging and audit archives

**The constraint.** Log volume × retention is a storage bill and a query latency; and modern practice already rejected JSON logs once — that is what **logfmt** (`level=info msg="..." dur=23ms`) is.

**The fit.** TASS is logfmt taken one step further: single-char keys via a dictionary, codes for enums, and — the part logfmt lacks entirely — a canonical form with in-band MACs (Ch. 8), which turns an application log into an *audit log* whose lines can be re-verified years later (`signed_audit_trail.py` demonstrated exactly this for telemetry).

`snippets/tass_logger.py` implements the whole idea as a stdlib `logging.Formatter`:

```
handler.setFormatter(TASSLogFormatter(LOG_SCHEMA))
log.info("checkout", extra={"user": "u_812", "route": "/pay", "ms": 231})
# emits: ~t:1751888100 ~l:i ~e:co ~u:u_812 ~r:/pay ~d:231
```

The "store compact, view expanded" pattern (Ch. 9 §9.6) was designed for this: `grep/awk` operate on raw lines; the dashboard expands through the dictionary at view time. Measured on the snippet's synthetic workload, TASS lines run ~40% smaller than equivalent JSON log lines — compounding across every replica, shipper, index, and retention day.

## 11.6 URL state and deep links

**The constraint.** URLs face practical length limits, aggressive truncation in chat apps and email clients, and human eyes.

**The fit.** Filter state, referral context, pagination cursors — `?q=~s:electronics+~x:100+~y:500+~o:pd` is shorter than its query-param equivalent, one *value* rather than N parameters (survives frameworks that mangle repeated params), and order-free (two users' links to the same view canonicalize to the same string — a cache key, via `hash_record`). Percent-encoding cost is minimal since the alphabet is already URL-safe ASCII apart from

the space separator (encode as `+`). For share-links that must not be forged — signed discount state, signed report views — the `~!` tag rides along.

## 11.7 Human-mediated records: field data collection

**The constraint.** Clipboard-and-biro digitization: agricultural surveys, cold-chain checks, utility meter reads. The scarce resources are transcription effort and error rate, not bytes.

**The fit.** A TASS line is *dictate-able and typeable*: "tilde-t 23.4, tilde-m 31, tilde-s ok" — and the parser's validation catches the two classic transcription failures (missing field: F3 by name; malformed pair: dropped, not guessed — Ch. 5 §5.8). Codes tables double as the enumerator's cheat card, which is printed on the *back of the clipboard*. This is the lowest-tech use case in the book and the one closest to classical stenography — a human writing shorthand against a shared dictionary — which is where the whole idea began.

## 11.8 The general selection rule

Six channels, one pattern. TASS fits a channel when **all four** hold:

1. Records are **flat and fixed-schema** (Ch. 3's boundary is unchanged).
2. The channel prices **characters or bytes** (or human effort) — and something in the loop is helped by the payload staying **printable ASCII**: a console, a human, an SMS gateway, a QR scanner, `grep`.
3. The **dictionary can travel out of band** — device docs, registry, clipboard card.
4. If the record will be *acted on*, the channel can afford `~22` extra characters for the `~!` tag.

Fail condition 2 and binary wins; fail 1 and JSON/CBOR's nesting wins; fail 3 and self-describing formats win. Everything else in this book — the registry, the ladder discipline (minus the LLM-specific rungs), the signing patterns, the observability stance — transfers to these channels without modification, because none of it ever depended on the tokenizer.

## 11.9 Summary

- Remove the LLM and TASS remains a compact printable record format for byte-, character-, and attention-priced channels: LPWAN telemetry, SMS/USSD/memo fields, QR/NFC payloads, structured logs, URL state, and field data capture.
- Against binary formats the pitch is explicit:  $\sim 1.5\text{--}2\times$  the bytes for zero toolchain and full `grep`-ability; where that trade is wrong, use the binary format.
- The dictionary-out-of-band and sign-at-the-trust-boundary disciplines transfer unchanged; only the budget arithmetic (tokens  $\rightarrow$  bytes) is new, and `payload_budgets.py` does it for every channel here.

*This closes the format's story arc: born from a tokenizer's pricing table, generalized to every channel that prices what it carries.*

## Appendix A — API Reference

Complete public API of `tass-protocol` 0.1.x. Everything below is importable from the top-level package:

```
from tass import (
    SchemaCompiler, TASSParser, TASSParseError, TASSValidationError,
    TASSFileParser, TASSFile, TASSFileError,
    TASSSigner, TASSIntegrityError, hash_record, derive_key, canonicalize,
)
```

### Core (`tass.parser`)

**`SchemaCompiler(prefix_char="~")`**

| Member               | Signature                                                     | Notes                                                                                                                                                                                                                                                                                                                           |
|----------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>compile</code> | <code>(schema: dict) -&gt; (parser_map, system_prompt)</code> | <code>schema</code> maps field name → <code>"string" \   "integer" \   "float" \   "boolean"</code> . Symbols assigned in declaration order from a 52-slot pool ( <code>a-z</code> , <code>A-Z</code> ); <code>ValueError</code> beyond 52 fields. <code>parser_map</code> maps symbol → <code>(field_name, type_hint)</code> . |

**`TASSParser(dictionary_map, prefix_char="~")`**

Accepts a `SchemaCompiler` map or a legacy `{symbol: field_name}` map (fields then default to string).

| Member                  | Signature                              | Notes                                                                                                                     |
|-------------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>parse</code>      | <code>(raw: str) -&gt; dict</code>     | Strips markdown, scans pairs, coerces types. Unknown symbols and non-pair tokens skipped.                                 |
| <code>safe_parse</code> | <code>(raw: str) -&gt; dict</code>     | <code>parse</code> + <code>validate</code> ; on failure retries as JSON; raises <code>TASSParseError</code> if both fail. |
| <code>validate</code>   | <code>(parsed: dict) -&gt; None</code> | Raises <code>TASSValidationError</code> naming missing fields.                                                            |

Coercions: `"18k" → 18000` (int), `"1.5k" → 1500.0` (float), `1/true/yes → True` (bool, case-insensitive).

**Exceptions:** `TASSParseError(ValueError)`, `TASSValidationError(ValueError)`.

## File format ( `tass.file_parser` )

### `TASSFileParser()`

| Member                  | Signature                                       | Notes                                                                                                 |
|-------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>parse</code>      | <code>(text: str) -&gt; TASSFile</code>         | Raises <code>TASSFileError</code> on unknown blocks or malformed <code>@dict / @codes</code> entries. |
| <code>parse_file</code> | <code>(path: str \  Path) -&gt; TASSFile</code> | UTF-8 read + <code>parse</code> .                                                                     |

### `TASSFile` (dataclass)

| Field                        | Type                    | Contents                                                                     |
|------------------------------|-------------------------|------------------------------------------------------------------------------|
| <code>dictionary</code>      | <code>dict</code>       | symbol → field name                                                          |
| <code>codes</code>           | <code>dict</code>       | code → full value (values may contain spaces)                                |
| <code>records</code>         | <code>list[dict]</code> | parsed records, codes expanded                                               |
| <code>raw_records</code>     | <code>list[str]</code>  | original wire lines (for hashing/signing)                                    |
| <code>to_parser_map()</code> | method                  | → <code>TASSParser</code> -compatible map (all types <code>"string"</code> ) |

**Exception:** `TASSFileError(ValueError)`.

## Integrity ( `tass.crypto` )

All symmetric, stdlib-only. Reserved signature symbol: `!` (constant `SIG_SYMBOL`).

| Function                  | Signature                                                                  | Notes                                                                 |
|---------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>canonicalize</code> | <code>(line, prefix_char="~") -&gt; str</code>                             | Pairs only, <code>~!</code> excluded, sorted by key, single-spaced.   |
| <code>hash_record</code>  | <code>(line, prefix_char="~") -&gt; str</code>                             | SHA3-256 hex (64 chars) of canonical form; order-independent.         |
| <code>derive_key</code>   | <code>(master_secret, context=b"", salt=b"", length=32) -&gt; bytes</code> | HKDF-SHA3-256 (RFC 5869).<br><code>ValueError</code> on empty secret. |

### `TASSSigner(key, tag_bytes=16, prefix_char="~")`

Key  $\geq 16$  bytes (32 recommended); `tag_bytes`  $\in [8, 32]$ , default 16 (128-bit tag → 22 base64url chars).

| Member                                     | Signature                        | Notes                                                                    |
|--------------------------------------------|----------------------------------|--------------------------------------------------------------------------|
| <code>sign_line</code>                     | <code>(line) -&gt; str</code>    | Appends <code>~!:&lt;tag&gt;</code> (HMAC-SHA3-256 over canonical form). |
| <code>verify_line</code>                   | <code>(signed) -&gt; bool</code> | Constant-time; <code>False</code> on absent/invalid tag. Never raises.   |
| <code>sign_records / verify_records</code> | <code>(list) -&gt; list</code>   | Element-wise batch forms.                                                |

**Exception:** `TASSIntegrityError(ValueError)` — provided for callers treating verification failure as fatal; the library itself returns bools.

## Benchmark (`tass.benchmark`) — requires `pip install tass-protocol[benchmark]` for exact counts

| API                                                                                   | Notes                                                                                                                                          |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TASSBenchmark(schema, records, encoding_name="o200k_base", approx=False)</code> | Renders TASS / pipe / minified-JSON / prose; <code>approx=True</code> (or missing <code>tiktoken</code> ) uses labelled whitespace estimation. |
| <code>.run() -&gt; list[BenchmarkResult]</code><br><code>.print_report()</code>       | <code>BenchmarkResult: format_name, tokens, vs_json_pct, sample.</code>                                                                        |
| <code>benchmark_from_tass_file(path, encoding_name=..., approx=...)</code>            | Schema inferred from <code>@dict</code> (string-typed).                                                                                        |

## CLI

```
tass compile <schema.json> [--map]
tass parse  "<record>" --schema <schema.json> [--safe]
tass read   <file.tass> [--records-only | --raw]
tass benchmark [<file.tass>] [--schema S --records R] [--encoding E] [--approx]
tass sign   "<record>" [--key-file F] [--context C]
tass verify  "<signed record>" [--key-file F] [--context C]
```

Key resolution for `sign/verify`: `--key-file` first, then `TASS_KEY` env var. Exit codes: `0` success/valid, `1` usage or parse error, `2` invalid signature.

## JavaScript (`tass.js`)

`SchemaCompiler#compile(schema) -> {parserMap, systemPrompt}; TASSParser({dictionaryMap, prefixChar})` with `parse(raw) / safeParse(raw)`. Mirrors the Python core (parser + compiler only; file format and crypto are Python-first as of 0.1.x).

# Appendix B — Glossary & Bibliography

---

## Glossary

**BPE (byte-pair encoding)** — Subword tokenization algorithm that greedily merges frequent byte pairs into a fixed vocabulary; the reason token cost equals corpus frequency, not visual length (Ch. 2).

**Canonical form** — A TASS record reduced to sorted, single-spaced `~key:value` pairs with the signature field removed; the input to all hashing and MAC operations (Ch. 8).

**Codes table (@codes)** — Declarative mapping from short wire codes to full values, expanded server-side after parsing (Ch. 6).

**Decode / prefill** — The two phases of LLM inference: parallel processing of the input (prefill) versus serial, per-token generation of the output (decode). Decode is why output tokens cost more (Ch. 1).

**Dictionary (@dict / parser map)** — The symbol → field mapping shared by prompt and parser, generated from one schema source (Ch. 4, 6).

**Fallback ladder** — The four-rung degradation strategy: `safe_parse` → `retry` → `JSON mode` → `dead-letter`; doubles as the pipeline health metric and circuit breaker (Ch. 5, 9).

**Grover's algorithm** — Quantum search with quadratic speedup; halves effective symmetric security bits, leaving 256-bit primitives with a  $\sim 2^{128}$  margin (Ch. 8).

**HKDF** — HMAC-based key derivation (RFC 5869); used with context strings to give each pipeline an independent signing key (Ch. 8).

**k-suffix** — Magnitude convention `18k = 18000`, `1.5k = 1500`; expanded by the parser's numeric coercion (Ch. 3).

**o200k\_base / cl100k\_base** — tiktoken encodings used by recent OpenAI model families; the reference vocabularies for this book's measurements (Ch. 7).

**Prompt caching** — Provider-side reuse of a repeated prompt prefix at  $\sim 90\%$  discount; the mechanism that amortizes the TASS dictionary to noise and imposes the byte-stable-prompt discipline (Ch. 4).

**Rung rate** — Fraction of requests resolved at each rung of the fallback ladder; the primary observability signal for a TASS pipeline (Ch. 9).

**Shor's algorithm** — Quantum algorithm breaking RSA/ECC; irrelevant to TASS's integrity layer because no public-key material exists in it (Ch. 8).

**Symbol pool** — The 52 single-token field markers `a-z`, `A-Z`; `!` is reserved for the integrity tag (Ch. 3, 8).

**TASS record** — One line of whitespace-separated `~symbol:value` pairs; order-free, self-delimiting, escaping-free (Ch. 3).

**TOON** — Token-Oriented Object Notation (2025); a token-reduced but still key-carrying and nesting-capable format; TASS's nearest published relative (Ch. 3).

## Bibliography & further reading

### Primary source

- Sharma, S. (2026). *Tokeniser-Aware Shorthand (TASS): A Stenography-Inspired Output Format for Reducing LLM Inference Cost in Structured Extraction Pipelines*. Zenodo. doi:10.5281/zenodo.20403219

### Tokenization

- Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL 2016 — the BPE paper.
- Gage, P. (1994). *A New Algorithm for Data Compression*. C Users Journal 12(2) — BPE's compression-era origin.
- OpenAI, *tiktoken* — reference tokenizer implementation. <https://github.com/openai/tiktoken>

### Cryptography (Ch. 8's standards trail)

- NIST FIPS 202 (2015). *SHA-3 Standard*.
- NIST SP 800-107r1 (2012). *Recommendation for Applications Using Approved Hash Algorithms* — HMAC truncation.
- Krawczyk, H., & Eronen, P. (2010). *RFC 5869: HKDF*.
- NIST FIPS 204 / 205 (2024). *ML-DSA / SLH-DSA* — the post-quantum signature schemes referenced for future non-repudiation support.
- Grover, L. (1996). *A Fast Quantum Mechanical Algorithm for Database Search*. STOC '96. · Shor, P. (1997). *Polynomial-Time Algorithms for Prime Factorization...* SIAM J. Comput. 26(5).

### Systems context

- Postel, J. (1980). *RFC 761*, §2.10 — the robustness principle behind the parser's liberal-scan/strict-validate split (Ch. 5).
- Provider documentation on prompt caching and batch inference — consult your provider's current docs; pricing figures in this book are illustrative and dated to early 2026.

### Format relatives

- RFC 822 header syntax; METAR aviation weather code — prior art for terse, schema-known channel formats (Ch. 3 §3.5).
- TOON — Token-Oriented Object Notation, 2025 (Ch. 3's comparison baseline for token-reduced structured formats).

## Appendix C — Use-Case Catalog

Every use case developed in this book and repository, with its complete implementation trail: the schema artifact, the code that exercises it, and the chapter that analyzes it. Each row is runnable today from the repository root.

### C.1 Support-ticket classification & routing

*The interactive workload: intent, urgency, sentiment, routing — per ticket, thousands of times a day. Latency savings matter as much as cost.*

| Artifact                                                 | Location                                                       |
|----------------------------------------------------------|----------------------------------------------------------------|
| Schema (registry-ready, with ~Z version pin)             | <code>examples/ticket_routing.tass</code>                      |
| Synchronous pipeline (compile → parse → validate → sign) | <code>snippets/end_to_end_pipeline.py</code>                   |
| Degradation strategy + circuit breaker                   | <code>snippets/fallback_ladder.py</code>                       |
| Versioned registry + version-skew demo                   | <code>snippets/schema_registry.py</code>                       |
| Analysis                                                 | Ch. 4 (compilation), Ch. 5 (failures), Ch. 9 Patterns I/III/IV |

```
tass read examples/ticket_routing.tass --records-only
python book/snippets/end_to_end_pipeline.py
```

### C.2 Influencer rate-card extraction

*The origin workload: 10-field commercial records where the LLM bill was a top-three cost line. Every value convention in one schema.*

| Artifact                           | Location                                             |
|------------------------------------|------------------------------------------------------|
| Schema + records                   | <code>spec/sample.tass</code>                        |
| Cost model (edit for your volumes) | <code>snippets/cost_model.py</code>                  |
| Analysis                           | Ch. 1 (economics), Ch. 3 (conventions), Ch. 10 §10.1 |

```
tass benchmark spec/sample.tass --approx
python book/snippets/cost_model.py
```

### C.3 Weather-API relay

*The adoption template: an LLM already summarizes an upstream API — move the output contract from provider-shaped JSON to a reduced TASS schema. Demonstrates schema reduction (~60 fields → 21) and multi-namespace codes tables (conditions, 16-point compass, AQI labels).*

| Artifact                       | Location                                                      |
|--------------------------------|---------------------------------------------------------------|
| Raw provider response (before) | <code>examples/weather_raw.json</code>                        |
| TASS schema + records (after)  | <code>examples/weather.tass</code>                            |
| Analysis                       | Ch. 6 (codes), Ch. 7 (the two honest baselines), Ch. 10 §10.2 |

```
tass read examples/weather.tass --records-only
tass benchmark examples/weather.tass --approx
```

## C.4 Quantum-device calibration telemetry

*The integrity workload: qubit coherence times, gate fidelities, and readout errors that downstream compilers act on — silent corruption is a wrong answer on hardware, not a logging bug.*

| Artifact                                              | Location                                          |
|-------------------------------------------------------|---------------------------------------------------|
| Schema + records (T1/T2, errors, lifecycle codes)     | <code>examples/quantum_telemetry.tass</code>      |
| Signed firehose, end to end (C.5's machinery applied) | <code>snippets/signed_audit_trail.py</code>       |
| Analysis                                              | Ch. 8 (integrity), Ch. 9 Pattern II, Ch. 10 §10.3 |

```
tass read examples/quantum_telemetry.tass --records-only
python book/snippets/signed_audit_trail.py
```

## C.5 Tamper-evident audit pipelines (the cryptographic use case)

*Not a schema but a capability that attaches to any of the above: sign at the trust boundary, ship raw signed lines, verify at every consumer, dedup by content address, re-verify archives years later.*

| Artifact                                                                                                             | Location                                          |
|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Library ( <code>TASSSigner</code> , <code>hash_record</code> , <code>derive_key</code> , <code>canonicalize</code> ) | <code>tass/crypto.py</code>                       |
| CLI                                                                                                                  | <code>tass sign</code> / <code>tass verify</code> |
| Working pipeline: tamper caught, replay deduped, reorder tolerated, archive re-verified                              | <code>snippets/signed_audit_trail.py</code>       |
| Threat model                                                                                                         | <code>docs/CRYPTO.md</code>                       |
| Analysis                                                                                                             | Ch. 8, Ch. 9 §9.3/§9.6                            |

```
export TASS_KEY="pipeline-secret"
tass sign "~a:rf ~b:5 ~c:1" --context tickets-v1
python book/snippets/signed_audit_trail.py
```

## C.6 Beyond the LLM: byte- and character-priced channels

Six non-LLM applications of the format itself — the budget unit becomes bytes, characters, or human effort instead of tokens. Full analysis with the honest "when binary formats win" boundary: Chapter 11.

| Use case                                                            | Artifact                                                              |
|---------------------------------------------------------------------|-----------------------------------------------------------------------|
| IoT / LPWAN sensor telemetry (LoRaWAN-class byte budgets)           | <code>examples/iot_sensor.tass</code>                                 |
| Structured / audit logging (stdlib <code>logging.Formatter</code> ) | <code>snippets/tass_logger.py</code>                                  |
| SMS / USSD / memo-field messaging, QR & NFC tags, URL state         | <code>snippets/payload_budgets.py</code> — exact fit-vs-budget tables |
| Field data collection (human-transcribed records)                   | Ch. 11 §11.7                                                          |

```
tass read examples/iot_sensor.tass --records-only
python book/snippets/tass_logger.py
python book/snippets/payload_budgets.py
```

## C.7 Choosing your own

Map a candidate workload onto the catalog by its dominant constraint:

| Your constraint looks like...                  | Start from | Because                                             |
|------------------------------------------------|------------|-----------------------------------------------------|
| Interactive latency, per-request calls         | C.1        | ladder + breaker are mandatory on user-facing paths |
| Raw cost at batch scale                        | C.2        | the cost model prices your exact schema first       |
| An upstream API you're already relaying        | C.3        | schema reduction is where most of the saving hides  |
| Data that downstream automation <i>acts on</i> | C.4 + C.5  | integrity is part of the schema, not an add-on      |

Then run the Chapter 9 §9.7 decision framework before committing — the five questions apply to every row above.